

计算机图形学原理

-----教程6

李瑞辉

大纲

- 多种的 种类 的 灯
- 复杂的 3D 对象
- 纹理贴图

-
- 方向光: 太阳
 - 点光源: 电灯泡
 - 点光: 手电筒

方向光:

当光源很远时, 我们能看全部这射线从它作为平行线。

所以我们定义光的方向向量而不是位置向量。

我们将方向定义为来自光源, 因此当光源位于顶部, 方向指向下方, 例如 $(-0.2f, -0.3f, -1.0f)$

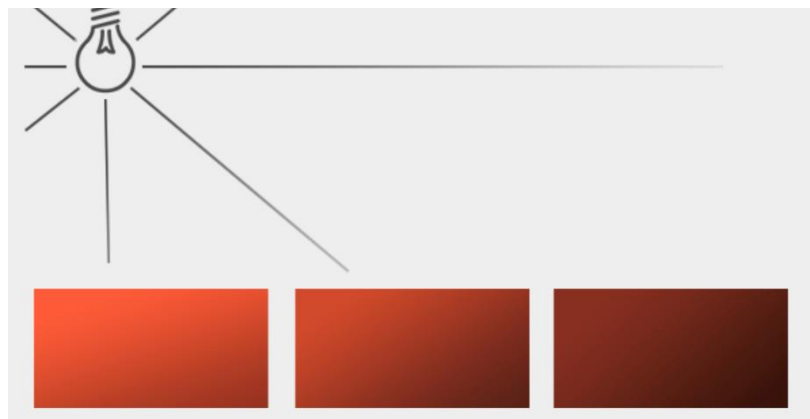
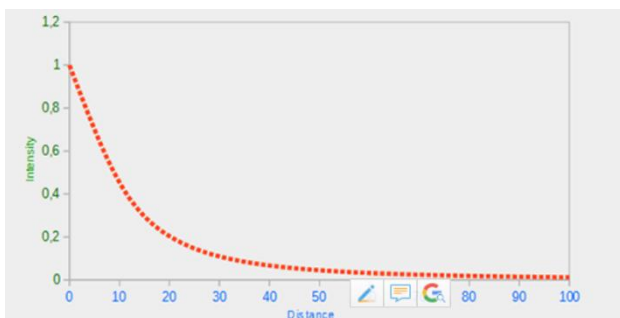
```
struct DirLight {  
    vec3 direction;  
    float intensity;  
  
    vec3 ambient;  
    vec3 diffuse;  
    vec3 specular;  
};
```

点光源:

特定位置的光源照亮所有方向，并随着距离逐渐减弱。

模拟衰减: K_c ，常数项; K_l ，线性项; K_q ，二次方学期。

$$F_{att} = \frac{1.0}{K_c + K_l * d + K_q * d^2}$$



经过 礼貌 的

<https://learnopengl.com/Lighting/Light-casters> __

选择正确的 K_c 、 K_l 和 K_q 参数:

Ogre3d's wiki: <http://wiki.ogre3d.org/tiki-index.php?page=-Point+Light+Attenuation>

```
▼ struct PointLight {  
    vec3 position;  
  
    float constant;  
    float linear;  
    float quadratic;  
  
    vec3 ambient;  
    vec3 diffuse;  
    vec3 specular;  
};
```

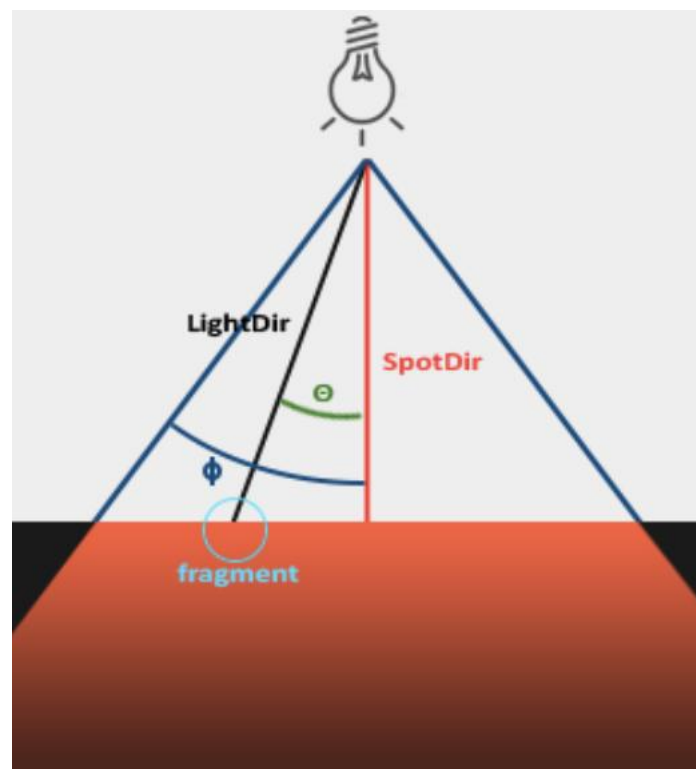
```
// attenuation  
float distance = length(light.position - fragPos);  
float attenuation = 1.0 / (light.constant + light.linear * distance + light.quadratic * (distance * distance));
```

点光： 自学

<https://learnopengl.com/Lighting/Light-casters>

光源只向特定方向发射光

```
struct SpotLight {  
    vec3 position;  
    vec3 direction;  
    float cutOff;  
    float outerCutOff;  
  
    float constant;  
    float linear;  
    float quadratic;  
  
    vec3 ambient;  
    vec3 diffuse;  
    vec3 specular;  
};
```



<https://learnopengl.com/Lighting/Light-casters>

将它们全部添加：

```
// phase 1: directional lighting
vec3 result = CalcDirLight(dirLight, norm, viewDir);

// phase 2: point lights
for (int i = 0; i < n_pointLight; i++)
    result += CalcPointLight(pointLight[i], norm, fragPos, viewDir);

// phase 3: spot light
for (int i = 0; i < n_spotLight; i++)
    result += CalcSpotLight(spotLight[i], norm, fragPos, viewDir);

fragColor = vec4(result, 1.0);
```

复杂的 3D 对象

➤ 当前版本OpenGL

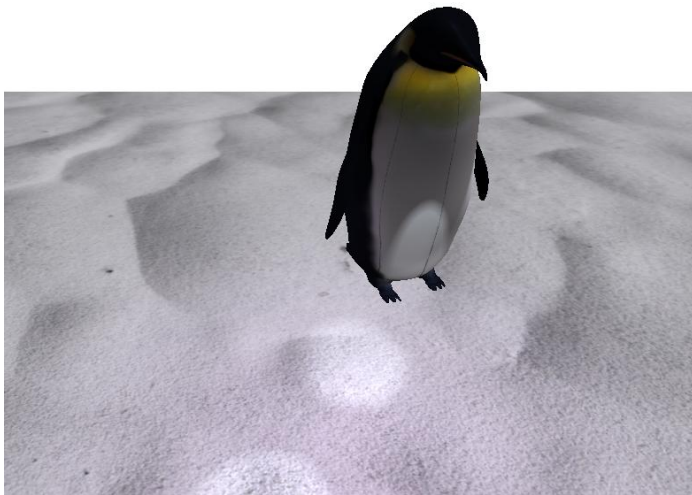
- 自己定义顶点属性。

```
const GLfloat plane[] =  
{  
    -5.0f, -1.0f, +5.0f,  
    +1.0f, +1.0f, +1.0f,  
  
    +5.0f, -1.0f, +5.0f,  
    +1.0f, +1.0f, +1.0f,  
  
    +5.0f, +1.0f, -15.0f,  
    +1.0f, +1.0f, +1.0f,  
  
    -5.0f, +1.0f, -15.0f,  
    +1.0f, +1.0f, +1.0f,};
```

```
const GLfloat cube[] =  
{  
    -1.0f, +1.0f, +1.0f, // 0  
    +1.0f, +0.0f, +0.0f, // Color  
  
    +1.0f, +1.0f, +1.0f, // 1  
    +0.0f, +1.0f, +0.0f, // Color.  
    .  
    .  
    .  
    +1.0f, -1.0f, +1.0f, // 23  
    +0.9f, +1.0f, +0.2f, // Color
```

复杂的 3D 对象

➤ 当前版本OpenGL



复杂的 3D 对象

➤ 当前版本OpenGL

- 通过.obj 文件加载复杂模型。
 - .obj 文件
 - 在哪里可以找到
 - 如何加载
 - 在 OpenGL 中使用

复杂的 3D 对象

➤ .obj文件

```
# Blender3D v249 OBJ File: untitled.blend
# www.blender3d.org
mtllib cube.mtl
v 1.000000 -1.000000 -1.000000
v 1.000000 -1.000000 1.000000
...
vt 0.748573 0.750412
vt 0.749279 0.501284
...
vn 0.000000 0.000000 -1.000000
vn -1.000000 -0.000000 -0.000000
...
usemtl Material_ray.png
s off
f 5/1/1 1/2/1 4/3/1
f 5/1/1 4/3/1 8/4/1
...
```

→ v: 顶点位置列表[x, y, z]

→ vt: UV坐标列表[u, v]

→ vn: 顶点法向量列表

→ f: face(三角信息)

复杂的 3D 对象

➤ .obj文件

f 5/1/1 1 / 2 /1 4 / 3 /1

第一个顶点:第二个顶点:第三个顶点:
A 乙 C

顶点法向量索引

f 5/1/1 1 / 2 /1

顶点位置索引

UV坐标指数

```
v 1.000000 -1.000000 -1.000000
v 1.000000 -1.000000 1.000000
...
vt 0.748573 0.750412
vt 0.749279 0.501284
...
vn 0.000000 0.000000 -1.000000
vn -1.000000 -0.000000 -0.000000
...
f 5/1/1 1/2/1 4/3/1
f 5/1/1 4/3/1 8/4/1
...
```

```
const GLfloat vertex[] = {
    1.00, -1.00, -1.00, // 1st
    position
    0.749279, 0.501284, //2nd
    UV coordinate
    0.00, 0.00, -1.00, //1st
    normal vector
}
```

复杂的 3D 对象

➤ 在哪里可以找到.obj文件

- 从网上下载

- <http://tf3dm.com/3d-models/all/1/obj>

- <http://www.yeggi.com/>

注意：需要改成合适的.obj格式

复杂的 3D 对象

- 自己用Blender画出复杂的模型然后导出

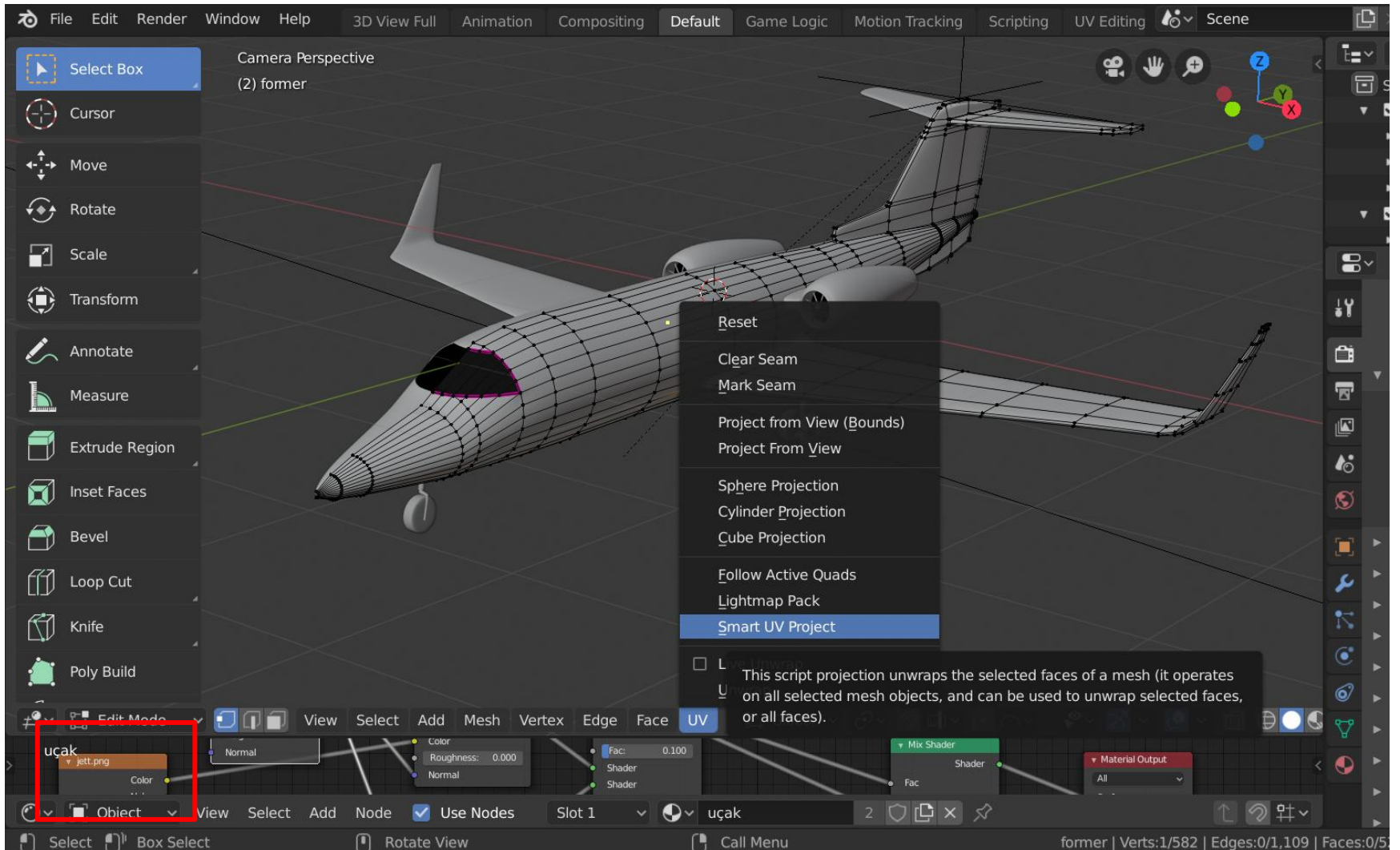
- 下载自

<https://www.blender.org/download/>

- 添加 UV 坐标

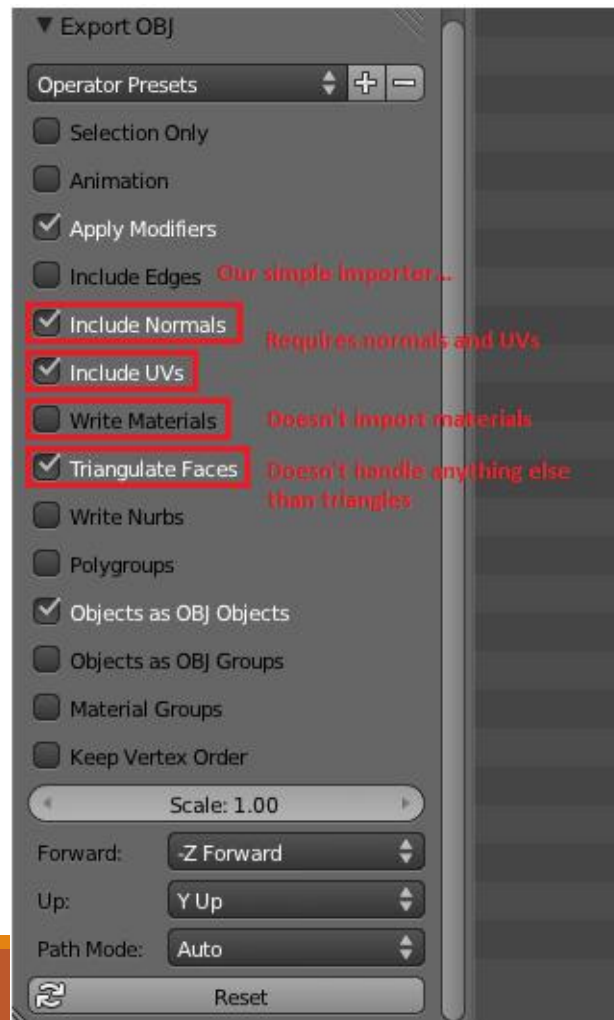
Edit-mode -> Press 'u' -> Select "Smart UV Project"

复杂的 3D 对象



复杂的 3D 对象

- 自己用Blender画出复杂的模型然后导出
 - 下载
 - 添加 UV 坐标
 - 导出



复杂的 3D 对象

➤ 加载.obj文件——**Model loadOBJ ()**函数

- 返回定义的 *模型* 结构
- 顶点结构应包含位置、*uv*坐标和法线信息。

```
struct Vertex { // struct for storing the obj file
    glm::vec3 position;
    glm::vec2 uv;
    glm::vec3 normal;
};
```

- 模型是包含许多顶点（*顶点结构*）的对象。

```
struct Model {
    std::vector<Vertex> vertices;
    std::vector<unsigned int> indices;
};
```

复杂的 3D 对象

➤ 加载.obj文件——**Model loadOBJ ()**函数

- 用于存储顶点信息的 **Temple** 变量

```
std::vector<glm::vec3> temp_positions;  
std::vector<glm::vec2> temp_uvs;  
std::vector<glm::vec3> temp_normals;  
std::map<V, unsigned int> temp_vertices;
```

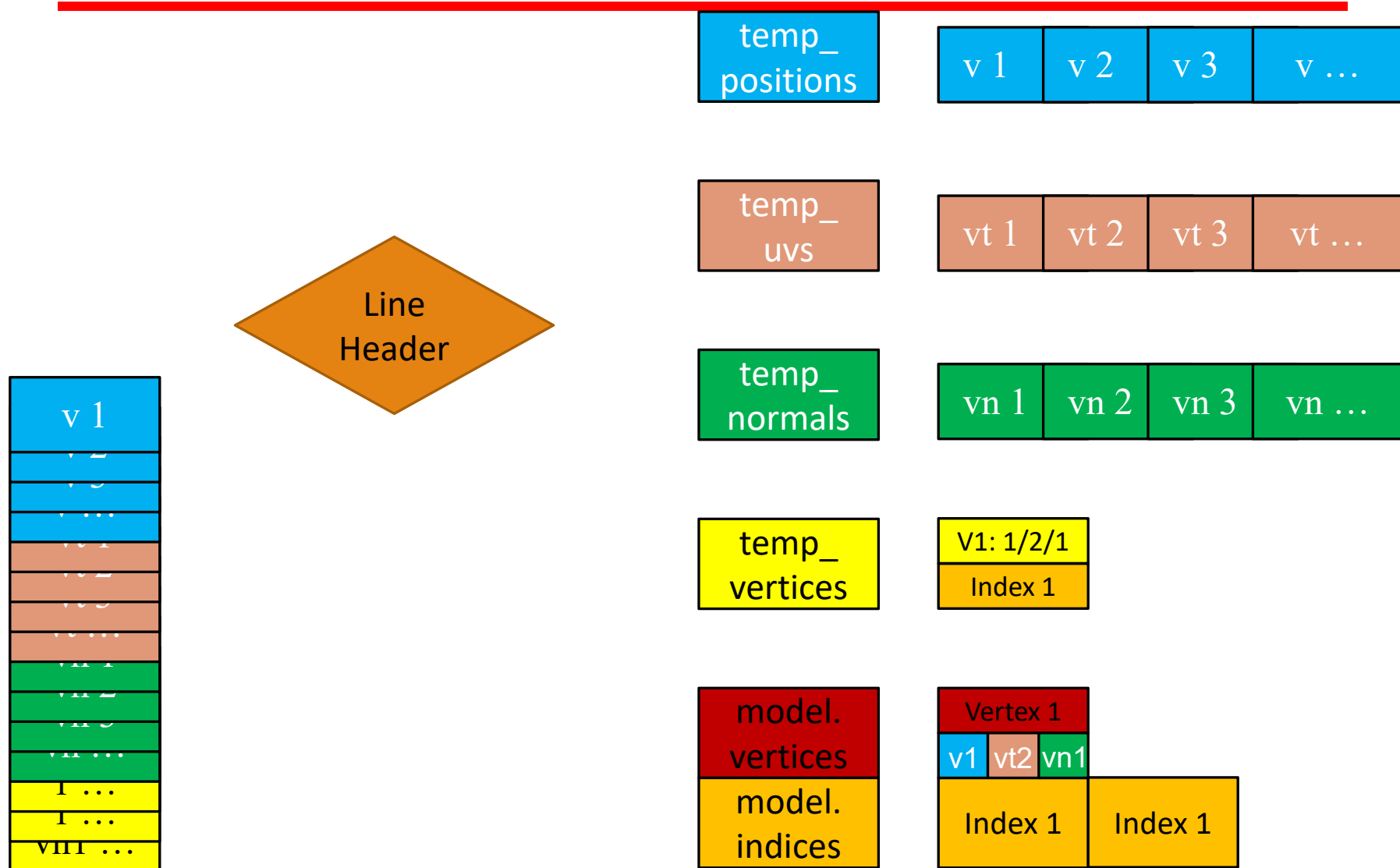
- 打开 .obj文件

```
struct V {  
    // struct for identify if a vertex has showed up  
    unsigned int index_position, index_uv, index_normal;  
    bool operator == (const V& v) const { ..... }  
    bool operator < (const V& v) const { ..... }  
};
```



f 5/1/1 1/2/1 4/3/1
face position index/ texture index/ normal index

复杂的 3D 对象



复杂的 3D 对象

➤ 加载.obj文件——**Model loadOBJ ()**函数

- 打开 .obj 文件

```
Model model; //returned value
unsigned int num_vertices = 0; // number of unique vertex

std::ifstream file;
file.open(objPath);

// Check for Error
if (file.fail()) {
    std::cerr << "Impossible to open the file! Do you use the
right path? See Tutorial 6 for details" << std::endl;
    exit(1);
}
```

复杂的 3D 对象

- 最后读取.obj文件

```
while (!file.eof()) {  
    // process the object file  
    char lineHeader[128];  
    file >> lineHeader;  
  
    ...  
}
```

复杂的 3D 对象

- 顶点位置

```
if (strcmp(lineHeader, "v") == 0) {  
    glm::vec3 position;  
    file >> position.x >> position.y >> position.z;  
    temp_positions.push_back(position);  
}
```

- 顶点UV坐标

```
} else if (strcmp(lineHeader, "vt") == 0) {  
    glm::vec2 uv;  
    file >> uv.x >> uv.y;  
    temp_uvs.push_back(uv);  
}
```

- 顶点法向量

```
} else if (strcmp(lineHeader, "vn") == 0) {  
    glm::vec3 normal;  
    file >> normal.x >> normal.y >> normal.z;  
    temp_normals.push_back(normal);  
}
```

复杂的 3D 对象

- face

```
} else if (strcmp(lineHeader, "f") == 0) {  
    // face elements  
    V vertices[3];  
    for (int i = 0; i < 3; i++) {  
        char ch;  
        file >> vertices[i].index_position >> ch >> vertices[i].index_uv >> ch >>  
vertices[i].index_normal;  
    }  
  
    // Check if there are more than three vertices in one face.  
    std::string redundancy;  
    std::getline(file, redundancy);  
    if (redundancy.length() >= 5) {  
        std::cerr << "Warning" << std::endl;  
        exit(1);  
    }  
    // to be continued with "f"
```

复杂的 3D 对象

- 更改为 OpenGL 可以使用的特定格式

```
// continued with "f"
for (int i = 0; i < 3; i++) {
    if (temp_vertices.find(vertices[i]) == temp_vertices.end()) {
        // the vertex never shows before
        Vertex vertex;
        vertex.position = temp_positions[vertices[i].index_position - 1];
        vertex.uv = temp_uv[vertices[i].index_uv - 1];
        vertex.normal = temp_normals[vertices[i].index_normal - 1];

        model.vertices.push_back(vertex);

        model.indices.push_back(num_vertices);
        temp_vertices[vertices[i]] = num_vertices;
        num_vertices += 1;
    }
    else { // reuse the existing vertex
        unsigned int index = temp_vertices[vertices[i]];
        model.indices.push_back(index);
    }
} // for
} // else if
```

复杂的 3D 对象

➤ 在 OpenGL 中使用 (void sendDataToOpenGL ())

- 加载 .obj 文件

```
// 读取 .obj 文件  
model obj = loadOBJ ("cube.obj");
```

- 创建顶点数组对象 (VAO) [一个模型一个 VAO]

```
glGenVertexArrays (1, & VertexArrayID );  
glBindVertexArray ( VertexArrayID );
```

复杂的 3D 对象

➤ 在 OpenGL 中使用 (void sendDataToOpenGL ())

- 创建顶点缓冲区对象 (VBO) [一个模型一个 VBO]

```
glGenBuffers (1, &VBO);  
glBindBuffer (GL_ARRAY_BUFFER, VBO);  
glBufferData (GL_ARRAY_BUFFER, obj.vertices.size () * sizeof ( Vertex ), &  
obj.vertices [0], GL_STATIC_DRAW);
```

Vertex position

UV

Normal

- 创建元素数组缓冲区对象 (EBO) [One model One EBO]

```
glGenBuffers (1, &EBO);  
glBindBuffer (GL_ELEMENT_ARRAY_BUFFER, EBO);  
glBufferData (GL_ELEMENT_ARRAY_BUFFER, obj.indices.size () * sizeof (  
 unsigned int ), & obj.indices [0], GL_STATIC_DRAW);
```

复杂的 3D 对象

➤ 发送到Shader (void paintGL (void))

- VBO发送给Vertexshader 【三属性一个VBO】

```
// 1st attribute buffer : position
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glEnableVertexAttribArray(0);
glVertexAttribPointer(
    0,                                // attribute
    3,                                // size
    GL_FLOAT,                          // type
    GL_FALSE,                          // normalized?
    sizeof(Vertex),                    // stride
    (void*)offsetof(Vertex, position)  // array buffer offset
);
```

- Draw

```
glDrawElements(GL_TRIANGLES, obj.indices.size(),
    GL_UNSIGNED_INT, 0);
```

复杂的 3D 对象

➤ **注意：** 仅当 .obj 文件的格式就像我们之前说的

```
# Blender3D v249 OBJ File: untitled.blend
```

```
# www.blender3d.org
```

```
mtllib cube.mtl
```

```
v 1.000000 -1.000000 -1.000000
```

```
v 1.000000 -1.000000 1.000000
```

```
...
```

```
vt 0.748573 0.750412
```

```
vt 0.749279 0.501284
```

```
...
```

```
vn 0.000000 0.000000 -1.000000
```

```
vn -1.000000 -0.000000 -0.000000
```

```
...
```

```
usemtl Material_ray.png
```

```
s off
```

```
f 5/1/1 1/2/1 4/3/1
```

```
f 5/1/1 4/3/1 8/4/1
```

```
...
```

→ v: 顶点位置[x,y,z]

→ vt : UV坐标[u, v]

→ vn : 顶点法向量

→ f: face(三角信息)

纹理贴图

```
# Blender3D v249 OBJ File: untitled.blend
# www.blender3d.org
mtllib cube.mtl
v 1.000000 -1.000000 -1.000000
v 1.000000 -1.000000 1.000000
```

```
...
vt 0.748573 0.750412
vt 0.749279 0.501284
...
```

vt : UV坐标[u, v]

用于纹理贴图

```
vn 0.000000 0.000000 -1.000000
vn -1.000000 -0.000000 -0.000000
```

```
...
usemtl Material_ray.png
s off
f 5/1/1 1/2/1 4/3/1
f 5/1/1 4/3/1 8/4/1
...
```

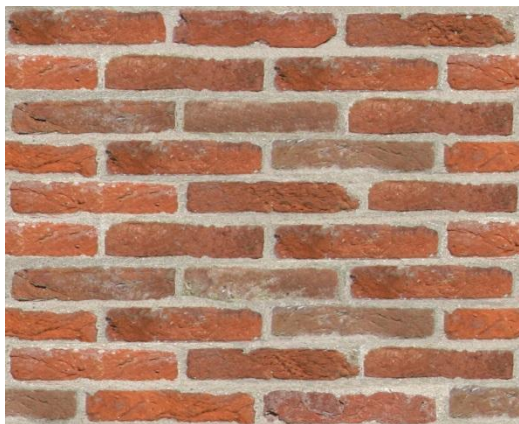
纹理贴图

➤ 什么是texture?

- 数据数组
- Texel（纹理像素）

➤ 为什么我们需要纹理？

- 例如



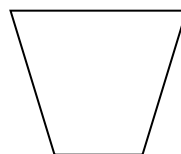
纹理贴图

- 将 1D、2D 或 3D 图像应用于几何图元
- 目的：增加 3D 对象的真实感
- 方法：将图像粘贴到 3D 模型的表面上。（减少几何复杂性、图像变形、反射等）



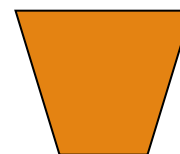
Texture

+



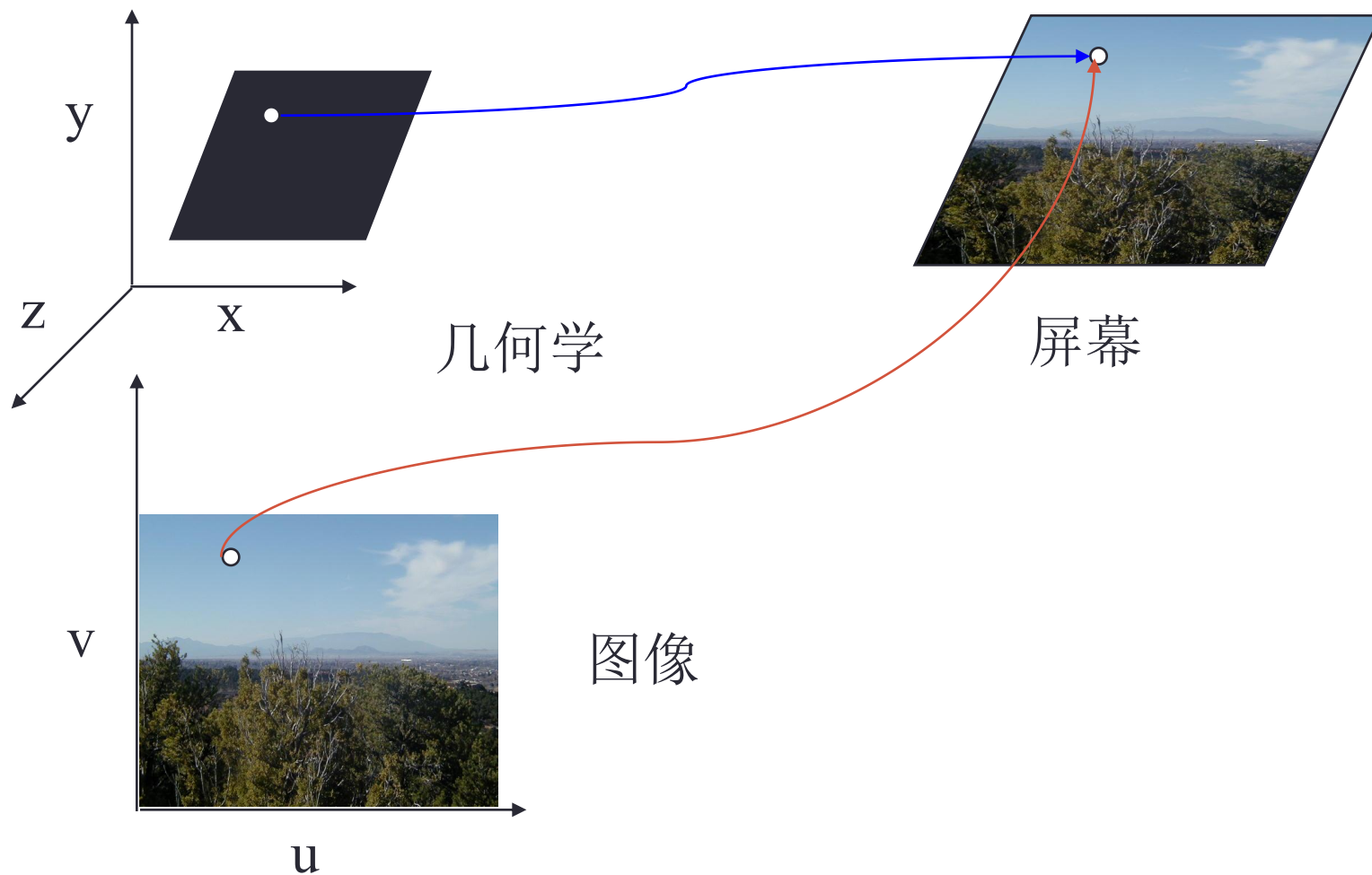
Polygon

=



**Textured-
Polygon**

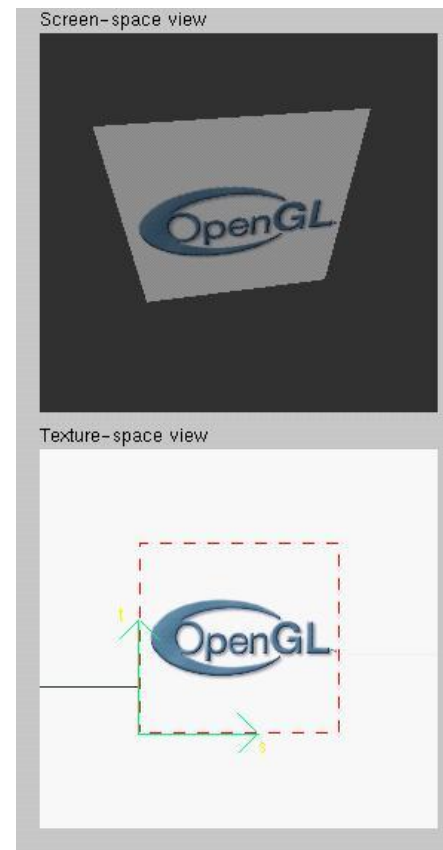
纹理贴图



纹理贴图

➤例子:

纹理是一个 256×256 的图像，已映射到以透视方式查看的矩形多边形



应用纹理

➤ 五个步骤

1. 加载 UV 坐标
2. 指定纹理_
 - 读取或生成图像
 - 分配给纹理
3. 指定纹理参数
 - wrapping, 过滤
4. 在 OpenGL 中加载
5. 着色器

UV坐标

➤加载 UV 坐标

- 我们加载的时候已经加载了.obj文件

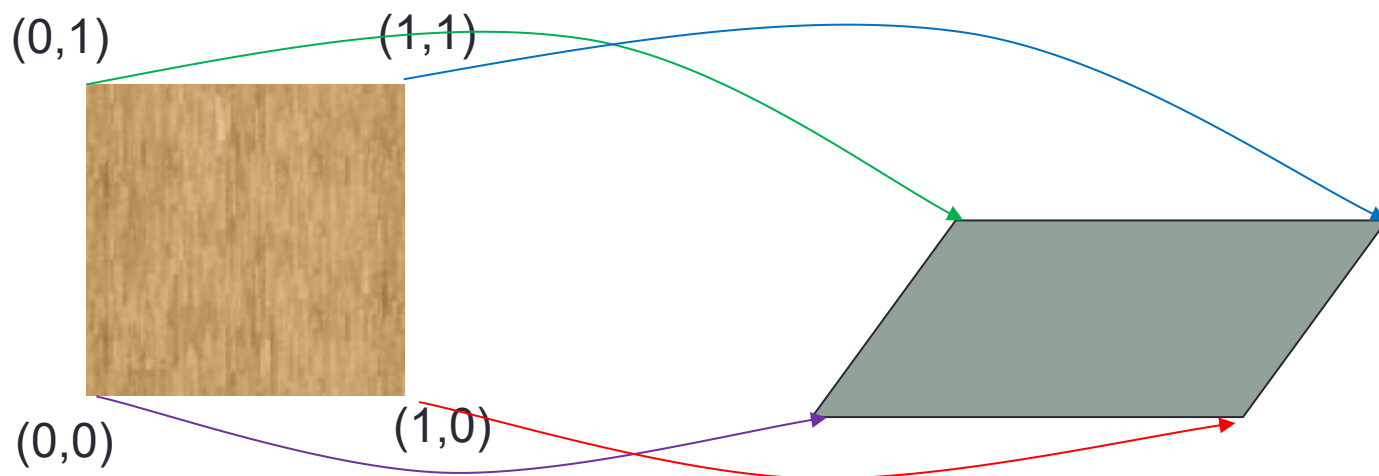
```
glEnableVertexAttribArray(1);
glVertexAttribPointer(
    1,                                // attribute
    2,                                // size
    GL_FLOAT,                          // type
    GL_FALSE,                          // normalized?
    sizeof(Vertex),                    // stride
    (void*)offsetof(Vertex, uv)        // array buffer offset
);
```

- 像其他属性一样使用

UV坐标

➤ 加载 UV 坐标

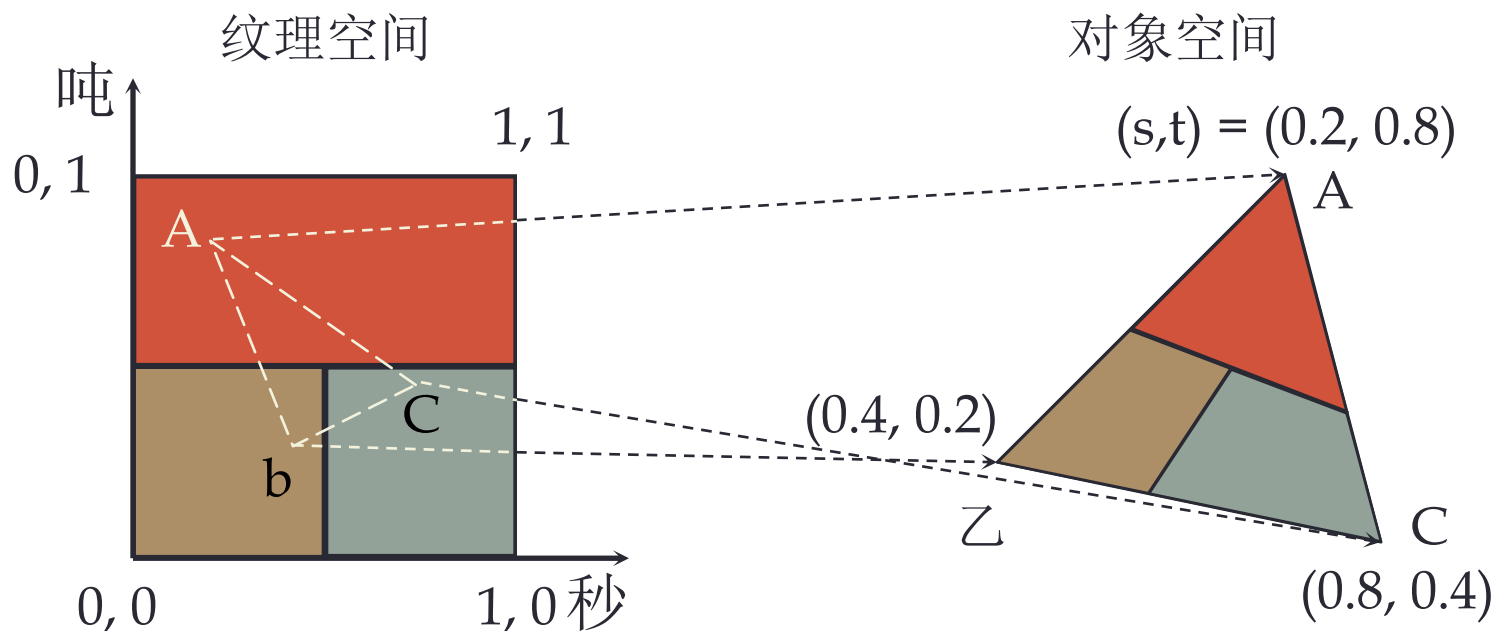
- 定义顶点和纹理的对应关系
- u, v 是要指定的纹理坐标



UV坐标

➤ 加载 UV 坐标

- 在每个顶点指定



应用纹理

➤ 五个步骤

1. 加载 UV 坐标
2. 指定纹理_
 - 读取或生成图像
 - 分配给纹理
3. 指定纹理参数
 - wrapping, 过滤
4. 在 OpenGL 中加载
5. 着色器

读取纹理图像

- 遗憾的是，OpenGL 不提供从文件系统加载图像的函数
- 我们必须使用一些其他函数来加载图像以进行纹理映射，例如（stb图像库）：

```
GLuint loadTexture(const char* texturePath)
```

- **注意：**
- 可以加载多种类型的图像
- 最好使用分辨率等于的纹理
2的幂



读取纹理图像——loadBMP_custom ()

```
GLuint loadTexture(const char* texturePath)
```

```
// tell stb_image.h to flip the loaded texture on the y-axis.
stbi_set_flip_vertically_on_load(true);
// load the texture data into "data"
int Width, Height, BPP;
unsigned char* data = stbi_load(texturePath, &Width, &Height, &BPP, 0);
GLenum format = 3;
switch (BPP) {
    case 1: format = GL_RED; break;
    case 3: format = GL_RGB; break;
    case 4: format = GL_RGBA; break;
}
```

- 我们~~已经~~提供了有关如何将图像文件加载到变量“**数据**”的代码，因此，您只需要知道如何将“**数据**”传递给 OpenGL

分配给纹理 - loadBMP_custom ()

```
GLuint loadTexture (const char* texturePath)
```

```
// Create one OpenGL texture
```

```
GLuint textureID;
```

```
glGenTextures(1, &textureID);
```

```
// "Bind" the newly created texture :
```

```
// to indicate all future texture functions will modify this texture
```

```
glBindTexture(GL_TEXTURE_2D, textureID);
```

```
// Give the image to OpenGL
```

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB,  
GL_UNSIGNED_BYTE, data);
```

```
// OpenGL has now copied the data. Free our own version
```

```
stbi_image_free(data);
```

➤ 示例代码：将数据传递给 OpenGL

应用纹理

➤ 五个步骤

1. 加载 UV 坐标
2. 指定纹理_
 - 读取或生成图像
 - 分配给纹理
3. 指定纹理参数
 - wrapping, 过滤
4. 在 OpenGL 中加载
5. 着色器

指定纹理参数

```
glTexParameteri(target, type, mode);
```

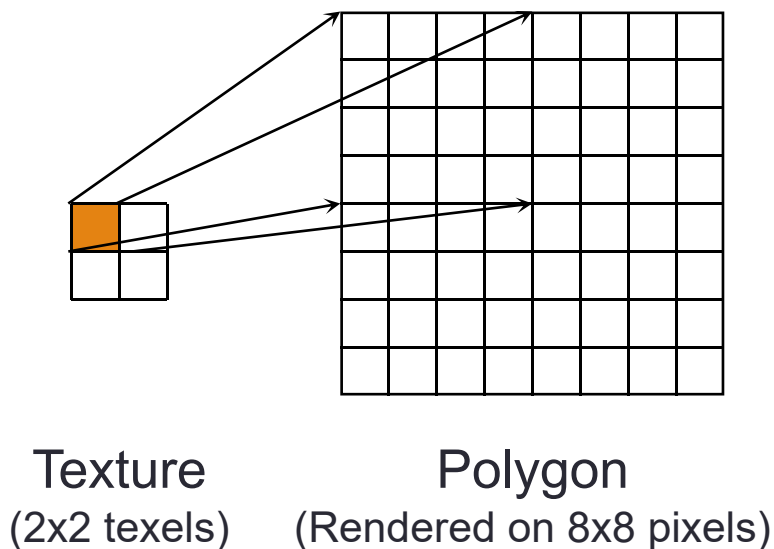
- 过滤模式
 - 缩小或放大
- 环绕模式
 - 夹紧或重复

指定纹理参数

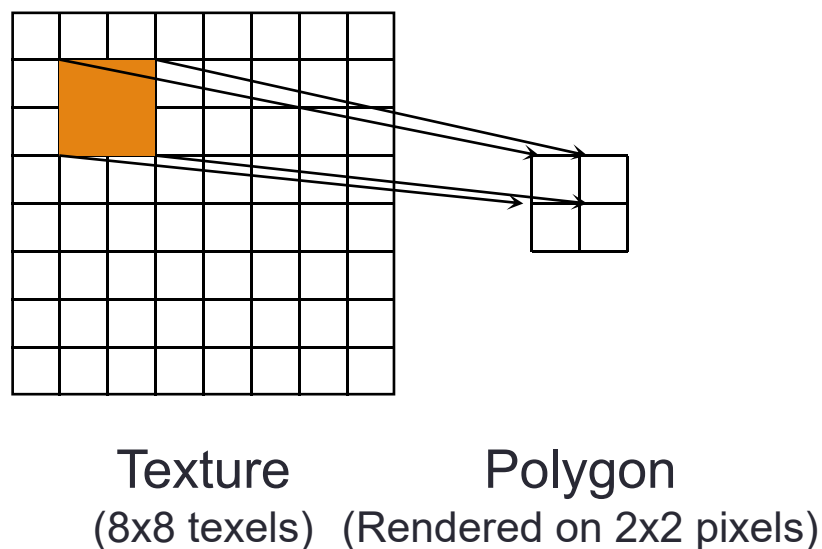
➤ 过滤模式

- 有时纹理贴图需要拉伸或收缩。
- 放大：每个纹素映射到一个以上的像素上。
- 缩小：每个纹素映射到少于一个像素上。

Magnification



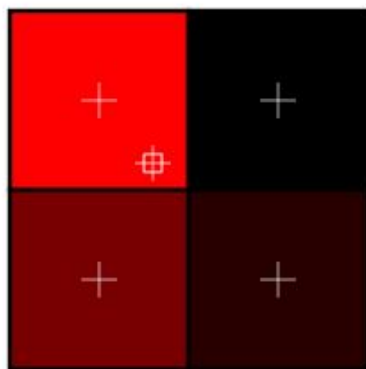
Minification



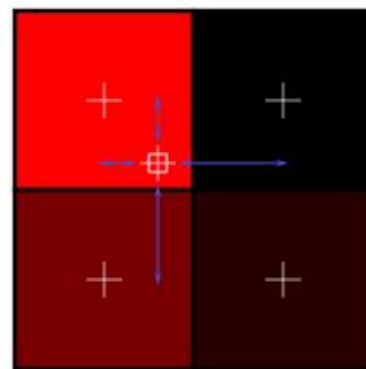
指定纹理参数

➤ 过滤模式

```
glTexParameteri ( GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);  
glTexParameteri ( GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```



GL_NEAREST
在 (u,v)处获取纹素
直接坐标

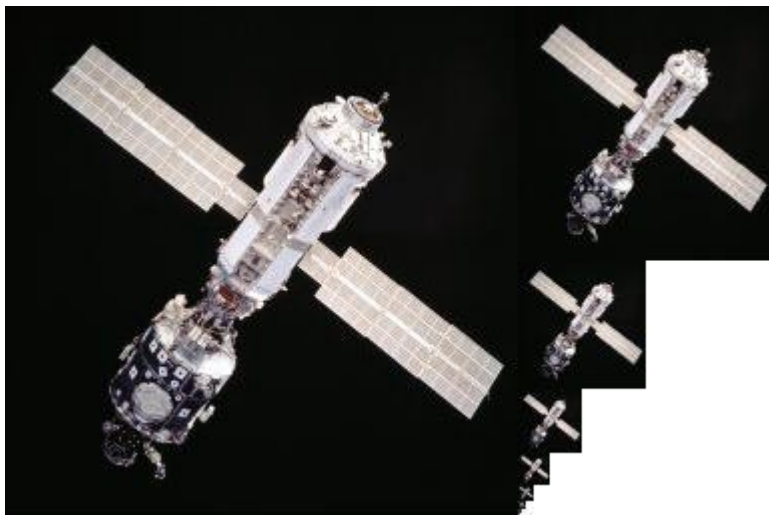


GL_LINEAR
考虑纹素在
邻域并计算
根据距离线性着色

指定纹理参数

➤ 过滤模式

```
glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,  
GL_LINEAR_MIPMAP_LINEAR);  
glGenerateMipmap (GL_TEXTURE_2D);
```



- 根据纹素大小
- mipmap

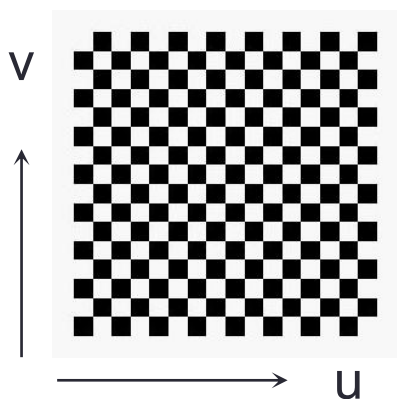
GL_LINEAR_MIPMAP_LINEAR
mipmap的线性混合，
每个也线性过滤

指定纹理参数

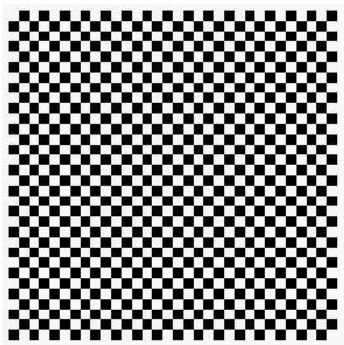
➤wrapping模式

```
glTexParameteri ( GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP );  
glTexParameteri ( GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT );
```

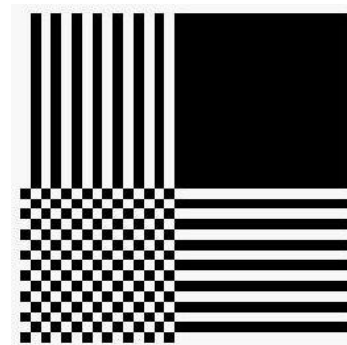
[0,1]范围之外分配纹理坐标，则让它们在纹理贴图中固定或重复。具有重复纹理，9 个副本平铺在一起。使用夹紧纹理，任何大于 1.0 的值都设置为 1.0，任何小于 0.0 的值都设置为 0.0。纹理的一个副本出现在中央。



texture



GL_REPEAT
wrapping



GL_CLAMP
wrapping

指定纹理参数——loadBMP_custom ()

```
GLuint loadTexture (const char* texturePath)
```

```
glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);  
glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);  
  
glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,  
GL_LINEAR);  
glTexParameteri (GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,  
GL_LINEAR_MIPMAP_LINEAR);  
glGenerateMipmap (GL_TEXTURE_2D);
```

➤ 示例代码：指定纹理参数

应用纹理

➤ 五个步骤

1. 加载 UV 坐标
2. 指定纹理_
 - 读取或生成图像
 - 分配给纹理
3. 指定纹理参数
 - wrapping, 过滤
4. 在 OpenGL 中加载
5. 着色器

在 OpenGL 中加载- sendDataToOpenGL ()

```
GLuint Texture0 = loadTexture ( “penguin_0 1 .jpg” );
```

➤ 示例代码：在 OpenGL 中加载

应用纹理

➤ 五个步骤

1. 加载 UV 坐标
2. 指定纹理_
 - 读取或生成图像
 - 分配给纹理
3. 指定纹理参数
 - wrapping , 过滤
4. 在 OpenGL 中加载
5. 着色器

Shader中应用——paintGL ()

➤ Shader中使用

```
unsigned int slot = 0  
GLuint TextureID = glGetUniformLocation(programID, "myTextureSampler0");
```

```
glActiveTexture(GL_TEXTURE0 + slot);  
  
glBindTexture(GL_TEXTURE_2D, Texture0);  
  
glUniform1i(TextureID, slot);  
  
...
```

活动纹理单元 0

在纹理单元 0 中绑定我们的纹理

将我们的 “ myTextureSampler ”
采样器（在片段着色器中）设置为用户纹理单元 0

如果您有多个纹理，请重复该过程。

Shader应用—VertexShader ()

➤ 获取UV坐标

```
in layout(location=1) vec2 vertexUV;  
  
out vec2 UV;  
  
int main()  
{  
    UV = vertexUV;  
}
```

Shader应用——FragmentShader ()

(1)从VertexShader获取UV坐标

```
in vec2 UV;
```

```
uniform sampler2D myTextureSampler0;
```

```
uniform sampler2D myTextureSmapler1;
```

```
int main()
```

```
{
```

```
    vec3 color = texture( myTextureSampler0, UV ).rgb;
```

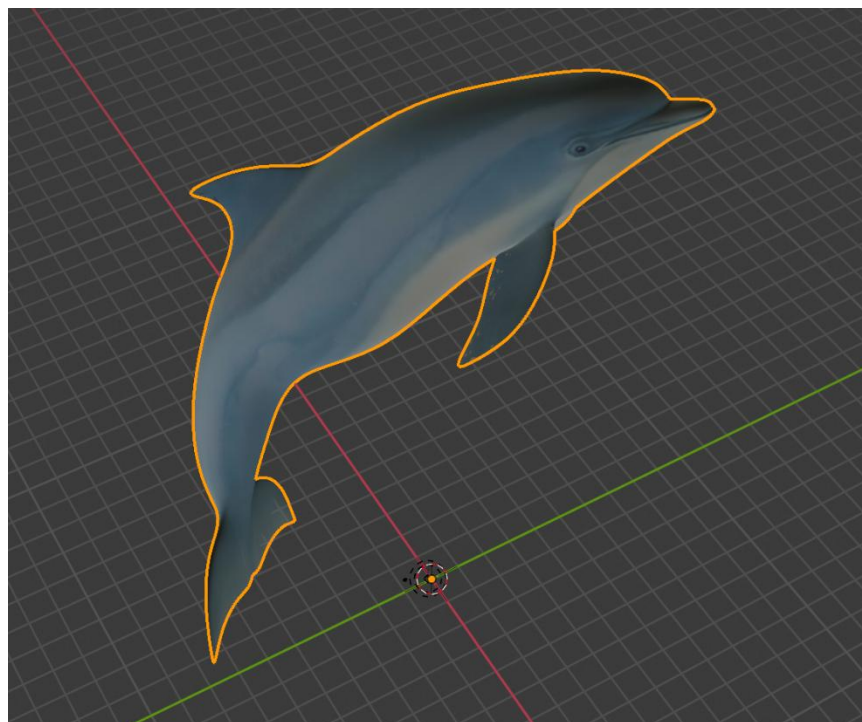
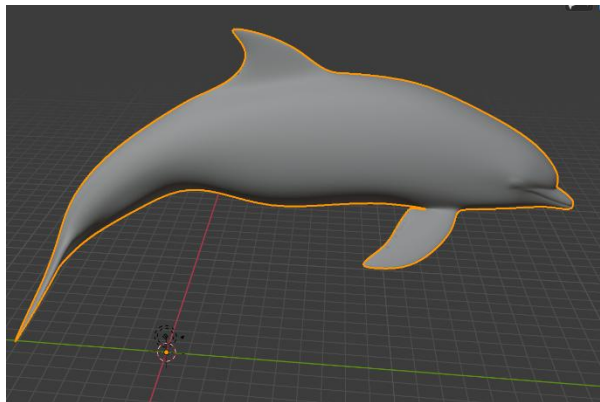
```
}
```

(2)从 OpenGL获取纹理

(3)计算

- texture()
- Return (R,G,B,A) vec4
- Just use R,G,B

纹理映射结果



下一教程

- 材质颜色
- 作业 2 中的基本要求

谢谢